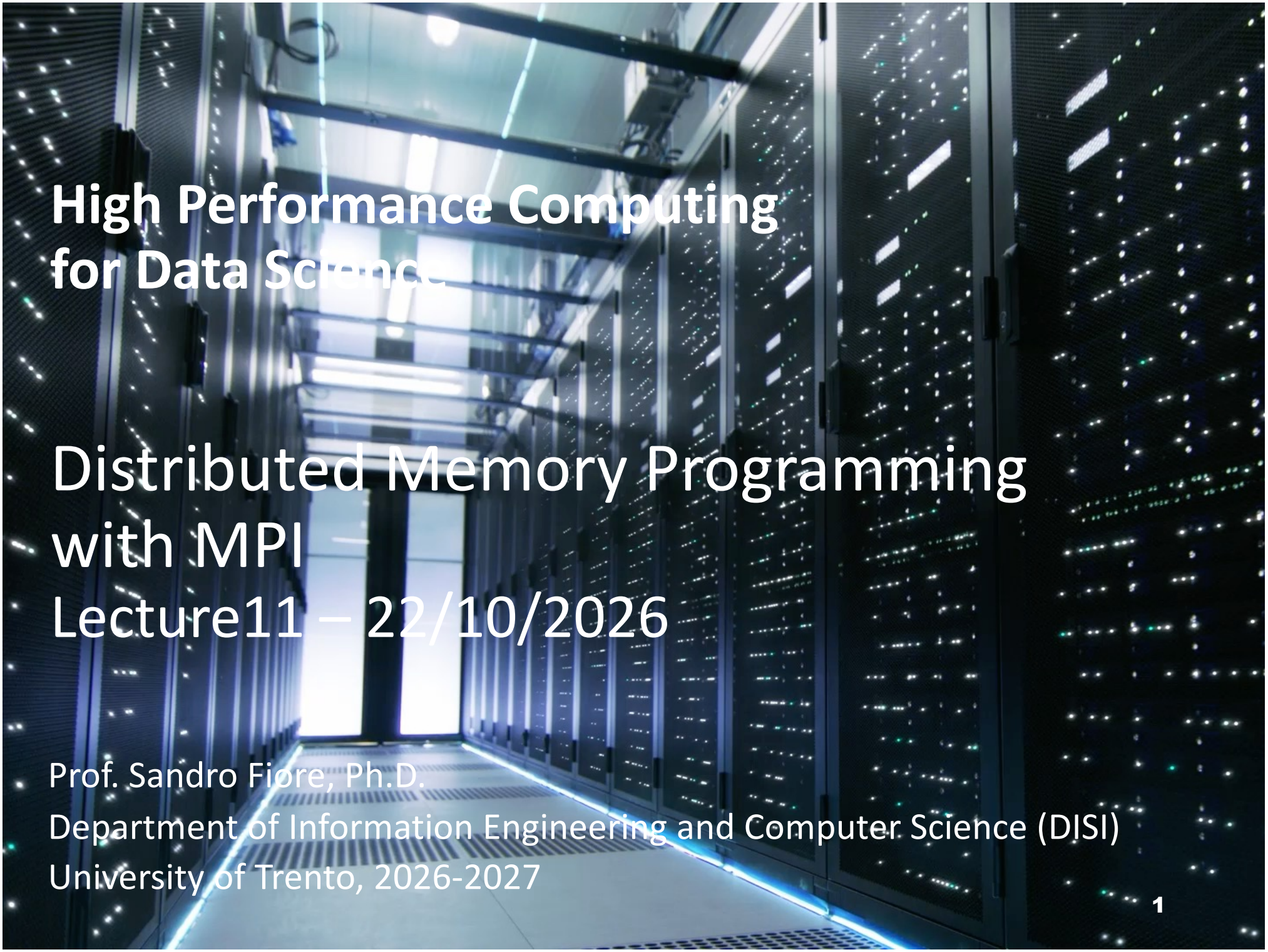


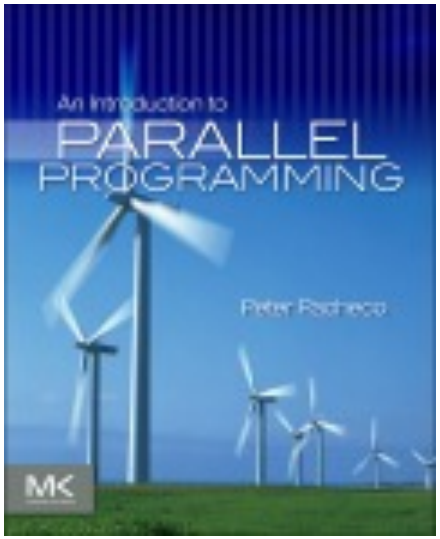
# High Performance Computing for Data Science

# Distributed Memory Programming with MPI
# Lecture11 – 22/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Today's outline and material

- Course projects and project template
- Recap:
  - Collective communications
  - Data Distribution
- PBS Placement Strategies




## Today's material

Book
*An Introduction to Parallel Programming by Peter Pacheco*

Chapter 3
*Distributed Memory Programming with MPI*

# International speakers

- Speakers from US asked to give a presentation in the Thu afternoons of November.
- These lectures are integral part of the course classes.

*Options:*

- *November 5th , 12th , 19th and 26th*
- December 3rd

Time slot:

- 2.30pm? Or 3.00? Or before?

# Project topics (I)

Parallel Bat Algorithm
Parallel solver per Aristotle's Number Puzzle
Parallel solver per Futoshiki puzzle (board at least 5 x 5, if possible 10 x 10)
Parallel Fast Hartley Transform
Parallel Bareiss algorithm
Parallel Levinson-Durbin algorithm
Parallel REQ (quantiles)
Parallel conjugate gradient
Parallel connected components
Parallel Q-Digest (quantiles)
Parallel Count-Min sketch
Parallel CPC (Compressed Probabilistic Counting) Sketch
Parallel Hyper Log Log Sketch
Parallel GSA (Gravitational Search Algorithm)
Parallel EM (Expectation-Maximization) clustering
Parallel CHARM (closed frequent itemsets mining)
Parallel Angle-Based Clustering
Parallel Selection
Parallel Spline Interpolation

# Project topics (II)

Parallel Hough Transform
Parallel Discrete Haar transform
Parallel Sinkhorn factorization of a  square matrix with positive entries
Parallel SVD factorization
Parallel Bareiss algorithm for computing the determinant of a matrix
Parallel Artificial Gorilla Troops Optimizer
Parallel Cholesky factorization of a dense, symmetric and positive definite square n x n matrix A
Parallel QR factorization
Parallel Power Method for eigenvalues
Parallel linear regression
Parallel SLASH
Parallel Matrix Inversion
Parallel CountSketch
Parallel tridiagonal linear system
Parallel numerical integration using Romberg's method
Parallel numerical integration using Gaussian Quadrature
Parallel Snyder Matrix Multiplication
Parallel Moth-Flame Optimization Algorithm
Parallel Raven Roosting Optimization Algorithm
Parallel root-finding of a polynomial

# Project template (I)

- **Introduction**. An accurate description of the assigned project should be provided, including analysis of the sequential algorithm that solves the problem addressed in the project.
  - Pseudo-code, examples, graphs, figures, application instances etc. may be provided.

- **Parallel design**. Preliminary study about the opportunities for parallelism inherent in the problem sequential algorithm.
  - State of the art analysis should be performed on parallel design strategies
  - Alternative designs, related to different parallelization strategies, should be considered and discussed at this stage, appropriately motivating why some operations lend themselves to effective parallelization and others do not as well as why some data structures may or may not minimize the burden of communication and synchronization.
  - Reference to prior work as well as link with related work must be clearly stated in the report.
  - Hybrid parallelization strategies, with data dependencies must be discussed too


Copyright © 2010, Elsevier Inc. All rights Reserved

# Project template (II)

- **Implementation**. C implementation (C++ is also allowed). The code must be properly commented. In case the student identifies multiple parallelization strategies, several implementations can be provided discussing pros and cons of each strategy. The report can include some code snippets related to the most critical and interesting parts.
  - A link to the repo must be provided too.
  - Hybrid parallelization is recommended though it is not mandatory

- **Performance and scalability analysis**.The student must analyze the performance of the developed implementation in terms of execution time, speedup, and efficiency.
  - Strong scalability and weak scalability should be evaluated where possible.
  - Single-node and multi-node benchmarks are expected


Copyright © 2010, Elsevier Inc. All rights Reserved

# Evaluation and significance of results

- The **evaluation** will also take into account:
  - Clarity and effectiveness of presentation;
  - Depth, correctness and originality of theoretical analysis;
  - Technical skills and documentation of implementation;
  - Number and quality of multiple parallel strategies, if any;
  - Critical thinking in performance evaluation and analysis;
- **Significance of results**: since parallel programming is primarily concerned with performance, a good project must also provide a significant speedup.
- Note: Expected size >=10 pages.
- AI statement in each report is mandatory.


Copyright © 2010, Elsevier Inc. All rights Reserved

# Relevant aspects for your project

- Cite references regarding papers or other material you got inspired by (state of the art analysis is part of the report!)
- Cite data sources you are using for your project
- Finding input data is challenging? Create a simple random generator app to create synthetic data
  - Different datasets (small, medium, large size) and then compare the performance of your application on the 3 input data
- Replicating/inspired by an existing work? Some tips
  - Reproduce and compare the results. Comment differences and other findings
  - Start from pseudocode and provide your C implementation; compare with findings from the authors
  - Add from existing code and add your own delta; then compare
  - Take 2 different implementations and port them on the cluster; compare the two on the cluster + original work

# Project and oral examination

- *Project*:
  - parallel application using MPI and OpenMP
  - to be developed in team
  - will be discussed during the oral examination
  - All the team members must be present during the oral examination and the project presentation

# High Performance Computing for Data Science

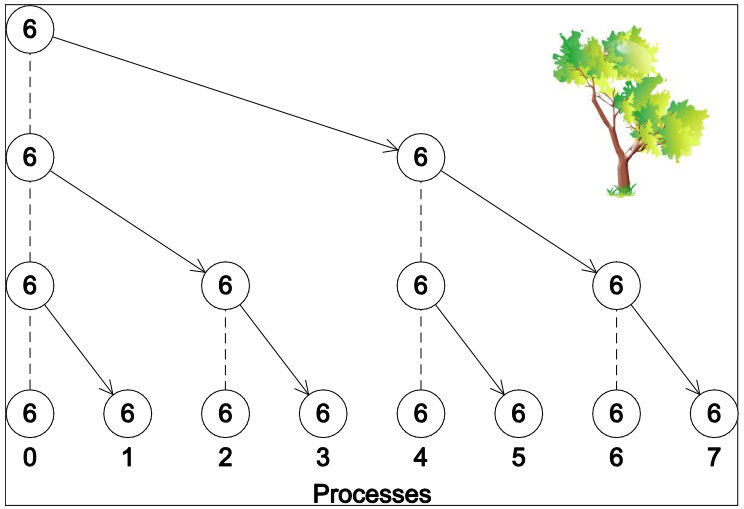
MPI Communication
Lecture11 - 22/10/2026

## Collective communications

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Broadcast

- Data belonging to a single process is sent to all of the processes in the communicator.




Tree structured implementation

```
int MPI_Bcast(
        void*         data_p      /* in/out */,
        int           count       /* in     */,
        MPI_Datatype  datatype    /* in     */,
        int           source_proc /* in     */,
        MPI_Comm      comm        /* in     */);
```


Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Reduce

```
int MPI_Reduce(
      void*         input_data_p   /* in  */,
      void*         output_data_p  /* out */,
      int           count          /* in  */,
      MPI_Datatype  datatype       /* in  */,
      MPI_Op        operator       /* in  */,
      int           dest_process   /* in  */,
      MPI_Comm      comm           /* in  */);
```

| Operation Value | Meaning |
|---|---|
| MPI_MAX | Maximum |
| MPI_MIN | Minimum |
| MPI_SUM | Sum |
| MPI_PROD | Product |
| MPI_LAND | Logical and |
| MPI_BAND | Bitwise and |
| MPI_LOR | Logical or |
| MPI_BOR | Bitwise or |
| MPI_LXOR | Logical exclusive or |
| MPI_BXOR | Bitwise exclusive or |
| MPI_MAXLOC | Maximum and location of maximum |
| MPI_MINLOC | Minimum and location of minimum |

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

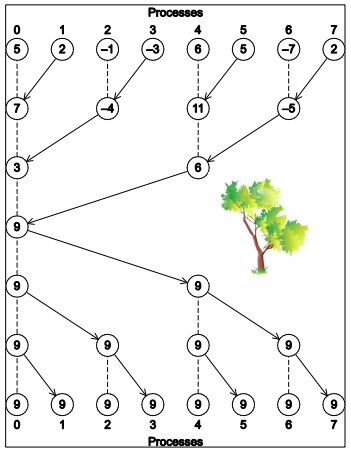
```
double local_x[N], sum[N];
. . .
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

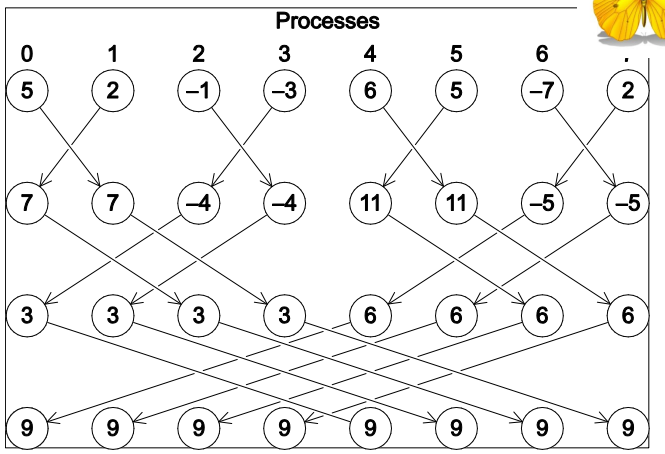
MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Allreduce

- Useful in a situation in which all of the processes need the result of a global sum in order to complete some larger computation.




Tree structured and buttefly

```
int MPI_Allreduce(
        void*        input_data_p   /* in  */,
        void*        output_data_p  /* out */,
        int          count          /* in  */,
        MPI_Datatype datatype       /* in  */,
        MPI_Op       operator       /* in  */,
        MPI_Comm     comm           /* in  */);
```

**"All" = "+ broadcast"**

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Collective vs. Point-to-Point Communications

- **All** the processes in the communicator must call the **same collective function**.
- The **arguments** passed by each process to an MPI collective communication **must be "compatible."**
- **Point-to-point** communications are matched on the basis of **tags** and **communicators**.
- **Collective** communications **don't use tags**.
- They're matched solely on the basis of the **communicator** and the **order** in which they're called.


Copyright © 2010, Elsevier Inc. All rights Reserved

# High Performance Computing for Data Science

MPI Communication
Lecture11 - 22/10/2026

## Data distribution: Scatter and Gather

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Scatter

- MPI_Scatter can be used in a function that reads in an entire vector on process 0 but only sends the needed components to each of the other processes.

```
int MPI_Scatter(
      void*          send_buf_p  /* in  */,
      int            send_count  /* in  */,
      MPI_Datatype   send_type   /* in  */,
      void*          recv_buf_p  /* out */,
      int            recv_count  /* in  */,
      MPI_Datatype   recv_type   /* in  */,
      int            src_proc    /* in  */,
      MPI_Comm       comm        /* in  */);
```


Copyright © 2010, Elsevier Inc. All rights Reserved

# Gather

- Collect all of the components of the vector onto process *dest_proc*, and then process *dest_proc* can process all of the components.

```
int MPI_Gather(
      void*          send_buf_p   /* in  */,
      int            send_count   /* in  */,
      MPI_Datatype   send_type    /* in  */,
      void*          recv_buf_p   /* out */,
      int            recv_count   /* in  */,
      MPI_Datatype   recv_type    /* in  */,
      int            dest_proc    /* in  */,
      MPI_Comm       comm         /* in  */);
```

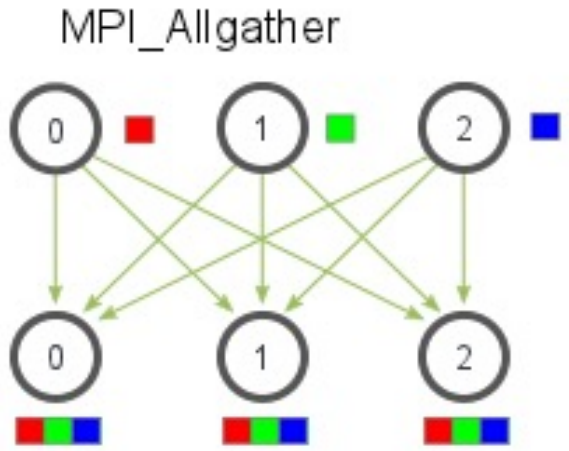
MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Allgather

- Concatenates the contents of each process' send_buf_p and stores this in each process' recv_buf_p.
- As usual, recv_count is the amount of data being received from each process.

```
int MPI_Allgather(
      void*          send_buf_p   /* in  */,
      int            send_count   /* in  */,
      MPI_Datatype   send_type    /* in  */,
      void*          recv_buf_p   /* out */,
      int            recv_count   /* in  */,
      MPI_Datatype   recv_type    /* in  */,
      MPI_Comm       comm         /* in  */);
```




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Matrix-vector multiplication

$A = (a_{ij})$ is an $m \times n$ matrix

$\mathbf{x}$ is a vector with $n$ components

$\mathbf{y} = A\mathbf{x}$ is a vector with $m$ components

$$y_i = a_{i0}x_0 + a_{i1}x_1 + a_{i2}x_2 + \cdots a_{i,n-1}x_{n-1}$$

*i-th component of y*

*Dot product of the ith row of A with x.*

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Multiply a matrix by a vector

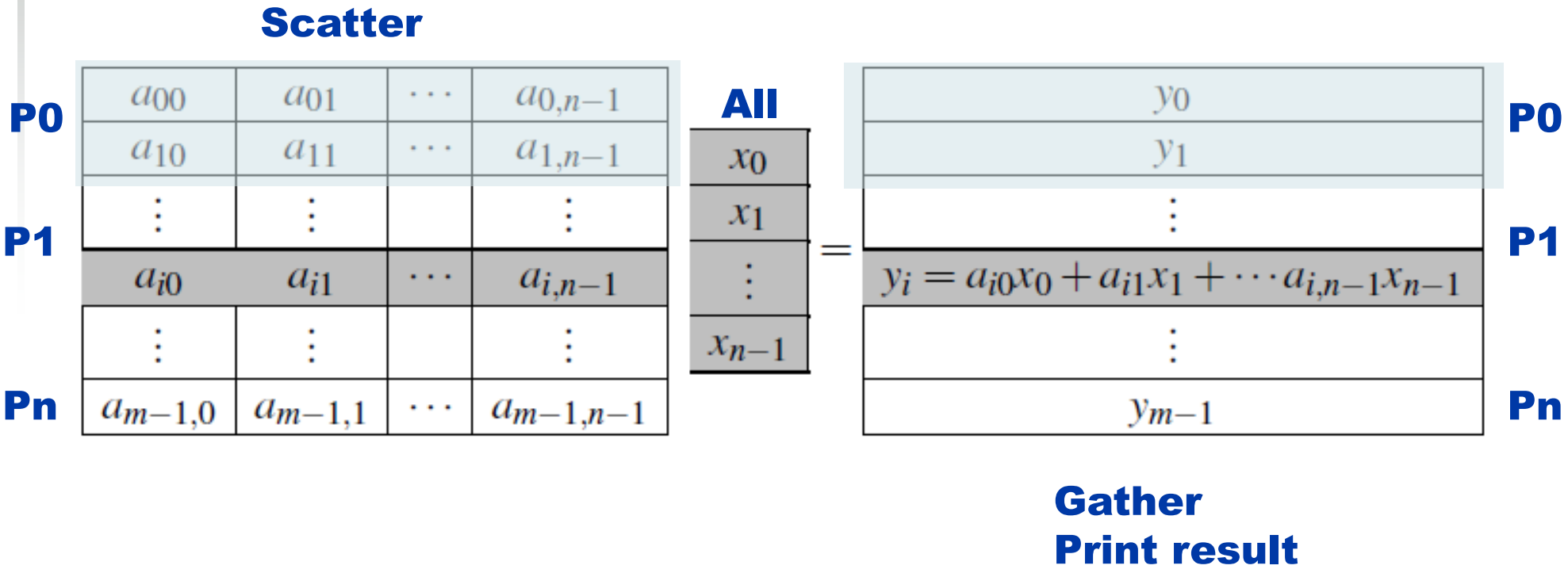
```
/* For each row of A */
for (i = 0; i < m; i++) {
    /* Form dot product of ith row with x */
    y[i] = 0.0;

    for (j = 0; j < n; j++)
        y[i] += A[i][j]*x[j];
}
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Matrix-vector multiplication




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# An MPI matrix-vector multiplication function

```
void Mat_vect_mult(
      double     local_A[]  /* in  */,
      double     local_x[]  /* in  */,
      double     local_y[]  /* out */,
      int        local_m    /* in  */,
      int        n          /* in  */,
      int        local_n    /* in  */,
      MPI_Comm   comm       /* in  */) {
   double* x;
   int local_i, j;
   int local_ok = 1;

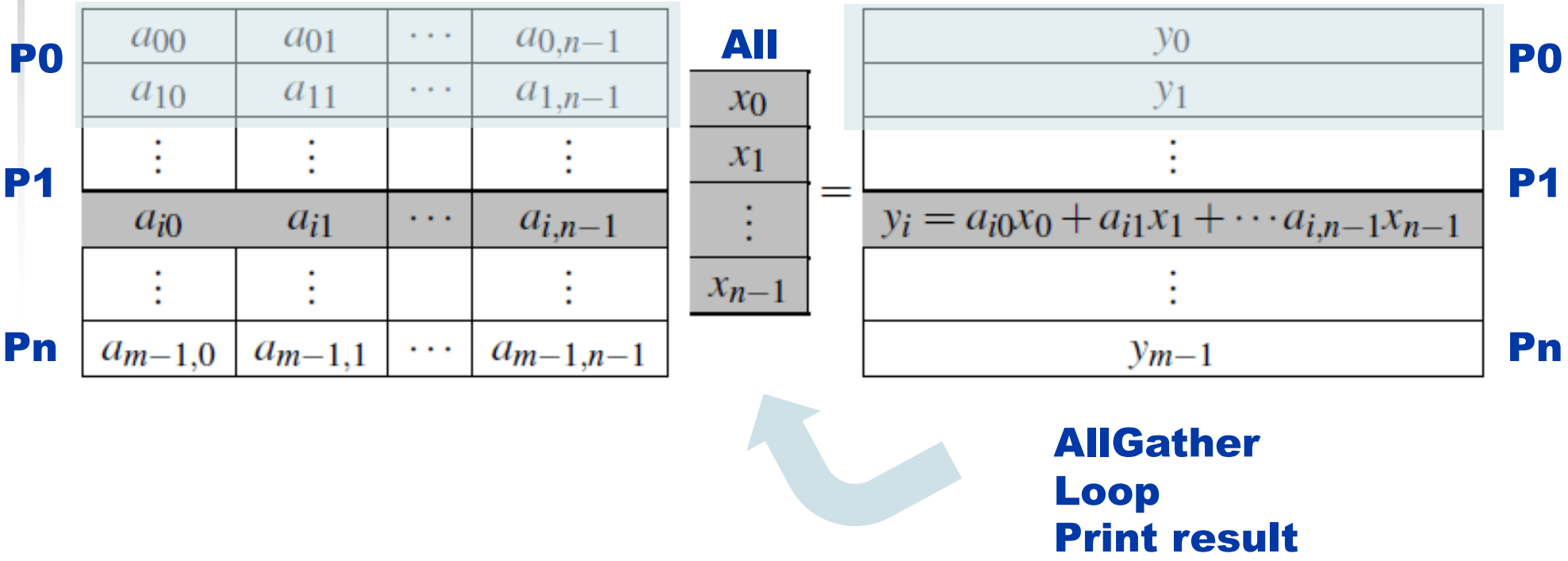
   x = malloc(n*sizeof(double));
   MPI_Allgather(local_x, local_n, MPI_DOUBLE,
         x, local_n, MPI_DOUBLE, comm);

   for (local_i = 0; local_i < local_m; local_i++) {
      local_y[local_i] = 0.0;
      for (j = 0; j < n; j++)
         local_y[local_i] += local_A[local_i*n+j]*x[j];
   }
   free(x);
}  /* Mat_vect_mult */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Matrix-vector multiplication




Feedback loop

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Closed ring exercise (solution)

```
#include <mpi.h>
#include <stdio.h>

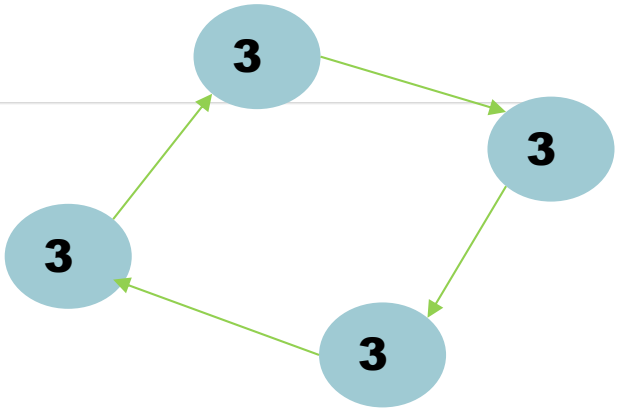
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int token;
    if (rank != 0) {
        MPI_Recv(&token, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received token %d from process %d\n", rank, token, rank - 1);
    } else {
        token = 2;
    }
    MPI_Send(&token, 1, MPI_INT, (rank + 1) % size, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        MPI_Recv(&token, 1, MPI_INT, size - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received token %d from process %d\n", rank, token, size - 1);
    }

    MPI_Finalize();
    return 0;
}
```




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Homework: MPI_Scatterv and MPI_Gatherv

- Learn about the difference between MPI_Scatter and MPI_Scatterv
- Discuss cases which need the use of the two calls


Copyright © 2010, Elsevier Inc. All rights Reserved

# Question time

# High Performance Computing for Data Science

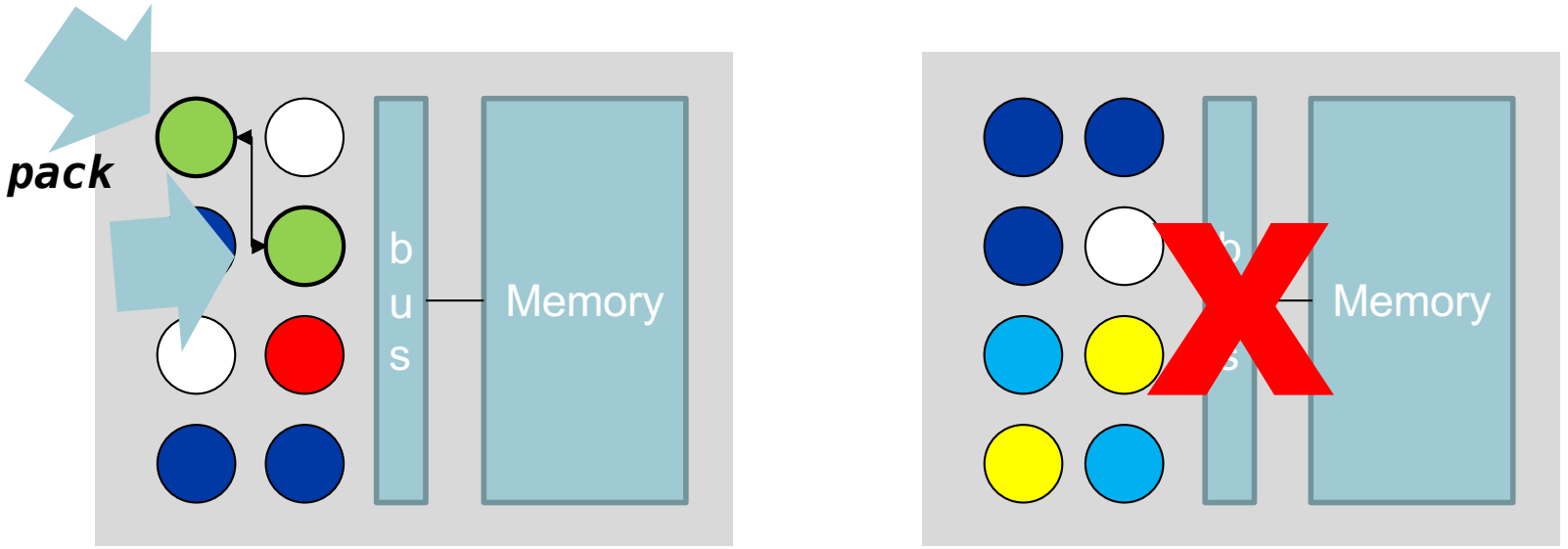
MPI Communication
Lecture11 - 22/10/2026

## PBS Placement Strategies

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# PBS Placing strategies (1)

1. #PBS –l select=2:ncpus=1:mem=2gb –l *place*=***pack***




Free core

Cores allocated to my app

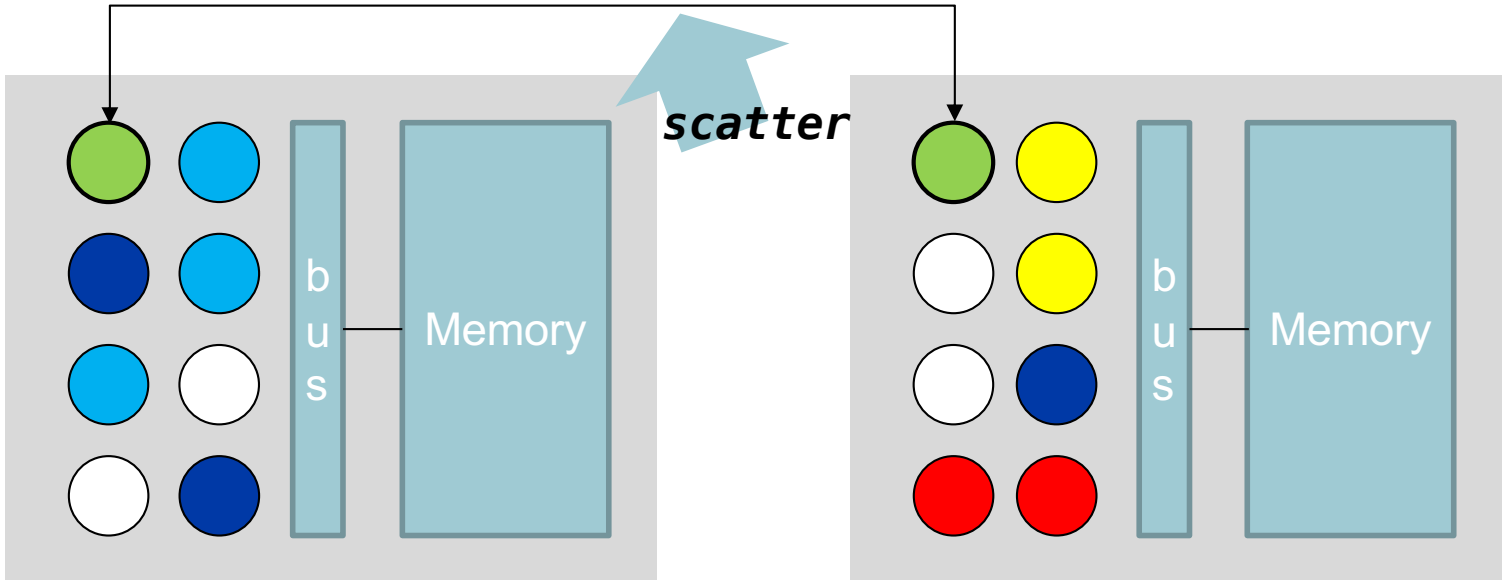
Cores allocated to other users' apps

***2 chunks (select 2) by 1 core (ncpus 1) on the same node (pack)***

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# PBS Placing strategies (2)

2. `#PBS -l select=2:ncpus=1:mem=2gb -l place=scatter`




Free core

Cores allocated to my app

Cores allocated to other users' apps

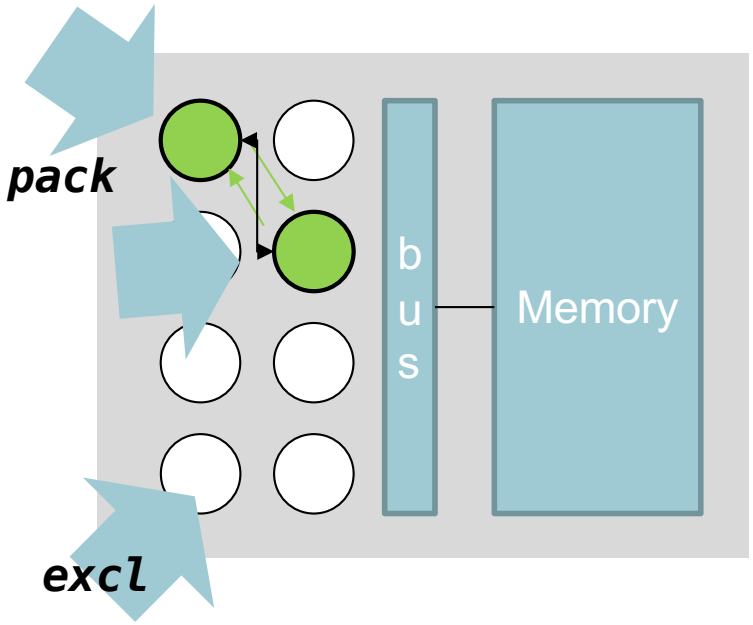
***2 chunks (select 2) by 1 core (ncpus 1) on two different nodes (scatter)***

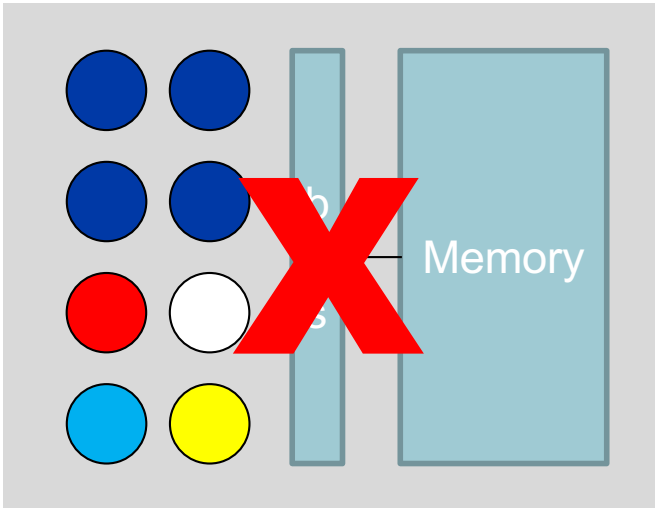
MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# PBS Placing strategies (3)

3. #PBS –l select=2:ncpus=1:mem=2gb –l *place=**pack:excl***




***2 chunks (select 2) by 1 core (ncpus 1) on the same node (pack) with exclusive allocation (excl)***

Free core

Cores allocated to my app

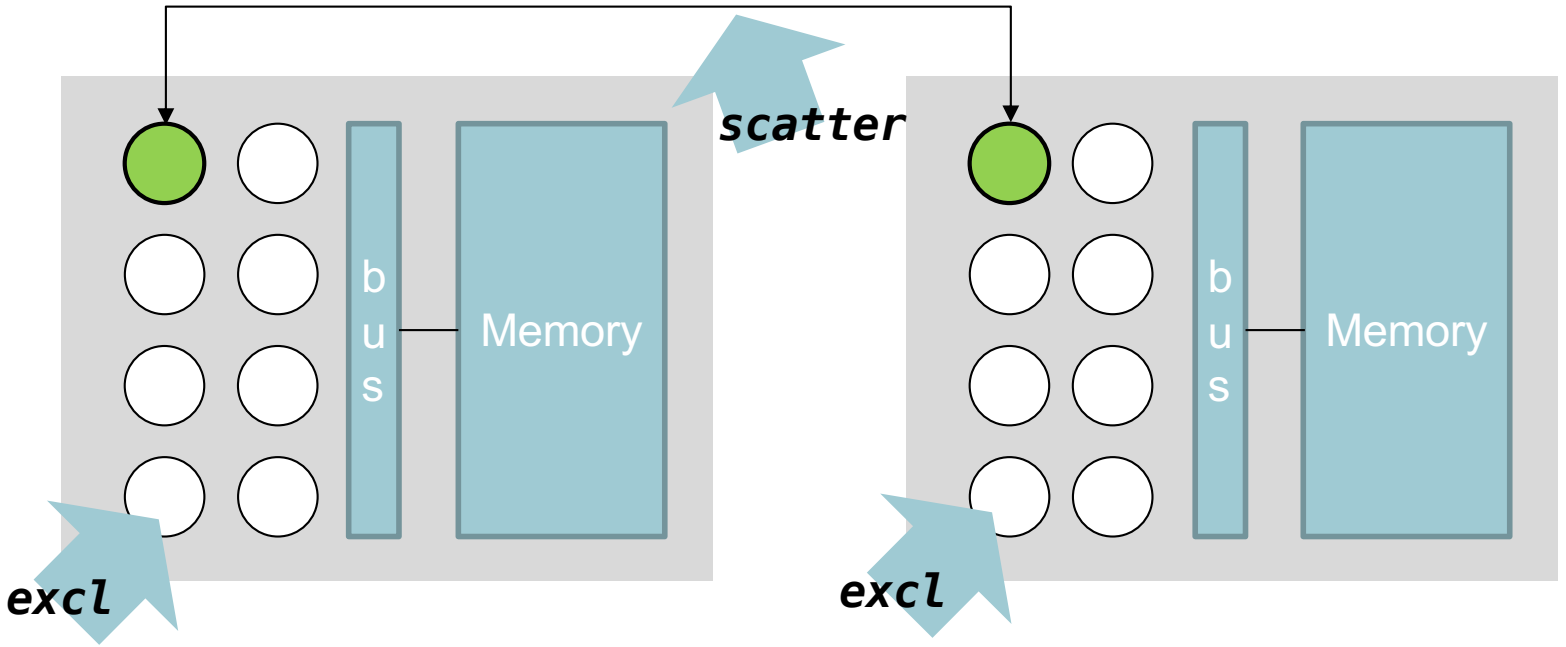
Cores allocated to other users' apps

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# PBS Placing strategies (4)

4. `#PBS -l select=2:ncpus=1:mem=2gb -l place=`***`scatter:excl`***




***2 chunks (select 2) by 1 core (ncpus 1) on two different nodes (scatter) with exclusive allocation (excl)***

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Parallel job submission

```
$ more test3.sh

#!/bin/bash

#PBS -l select=4:ncpus=4:mem=2gb

# set max execution time
#PBS -l walltime=0:00:30

# set the queue
#PBS -q shortCPUQ

module load MPICH/3.4.2-GCC-10.3.0

mpiexec -n 16 ./mpi_wrapper.sh ./ring
```





MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# mpi_wrapper.sh

```
#!/bin/bash

IFACE=$(ip -4 route get 192.168.108.1 |
        awk '{for (i=1;i<=NF;i++) if ($i=="dev") {print $(i+1);
exit}}')

if [[ -z "$IFACE" || "$IFACE" == "lo" ]]; then
    echo "ERROR: No valid Ethernet interface found on $(hostname)"
>&2
    exit 1
fi

export UCX_TLS=tcp,self,sm
export UCX_NET_DEVICES="$IFACE"

echo "$(hostname): UCX_NET_DEVICES=$UCX_NET_DEVICES"

exec "$@"
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Hands-on

- Complete the hands-on exercise assigned during Week4
- Run them both on single node and multi node configurations
- Learn about the gettimeofday and MPI_Wtime
- Use the different placing strategies and check the difference


Copyright © 2010, Elsevier Inc. All rights Reserved

# New MPI calls: MPI_Wtime

## MPI_Wtime

Returns an elapsed time on the calling processor

### Synopsis

```
double MPI_Wtime( void )
```

### Return value

Time in seconds since an arbitrary time in the past.

### Notes

This is intended to be a high-resolution, elapsed (or wall) clock. See `MPI_WTICK` to determine the resolution of `MPI_WTIME`. If the attribute `MPI_WTIME_IS_GLOBAL` is defined and true, then the value is synchronized across all processes in `MPI_COMM_WORLD`.

### Notes for Fortran

This is a function, declared as `DOUBLE PRECISION MPI_WTIME()` in Fortran.

See Also also: MPI_Wtick, MPI_Comm_get_attr, MPI_Attr_get

**Location:**src/mpi/timer/wtime.c

https://www.mpich.org/static/docs/v3.1/www3/MPI_Wtime.html


Copyright © 2010, Elsevier Inc. All rights Reserved

# gettimeofday

```
GETTIMEOFDAY(2)          FreeBSD System Calls Manual          GETTIMEOFDAY(2)

NAME
     gettimeofday, settimeofday -- get/set date and time

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <sys/time.h>

     int
     gettimeofday(struct timeval *tp, struct timezone *tzp);

     int
     settimeofday(const struct timeval *tp, const struct timezone *tzp);

DESCRIPTION
     The system's notion of the current Greenwich time and the current time
     zone is obtained with the gettimeofday() system call, and set with the
     settimeofday() system call.  The time is expressed in seconds and mi-
     croseconds since midnight (0 hour), January 1, 1970.  The resolution of
     the system clock is hardware dependent, and the time may be updated con-
     tinuously or in "ticks".  If tp or tzp is NULL, the associated time in-
     formation will not be returned or set.

     The structures pointed to by tp and tzp are defined in <sys/time.h> as:

     struct timeval {
             time_t          tv_sec;         /* seconds */
             suseconds_t     tv_usec;        /* and microseconds */
     };

     struct timezone {
             int     tz_minuteswest; /* minutes west of Greenwich */
             int     tz_dsttime;     /* type of dst correction */
     };

     The timezone structure indicates the local time zone (measured in minutes
     of time westward from Greenwich), and a flag that, if nonzero, indicates
     that Daylight Saving time applies locally during the appropriate part of
     the year.
```

# Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform